

« Automatiser, c'est transformer du temps humain répété
en logique exécutée une fois pour toujours. »

CE GUIDE COUVRE :

- PARTIE A → Automatisation des tâches
 - Fichiers & dossiers
 - Web & APIs
 - Emails & notifications
 - Bases de données
 - Navigateur & scraping
 - Planification & orchestration
- PARTIE B → Analyse de données
 - Chargement & exploration
 - Nettoyage
 - Transformation & feature engineering
 - Analyse statistique
 - Visualisation
 - Modélisation & interprétation

PARTIE A – AUTOMATISATION DES TÂCHES

A.1 – FICHIERS ET DOSSIERS

LIBRAIRIES ESSENTIELLES

pathlib → manipulation de chemins (modern, recommandé)
shutil → copie, déplacement, suppression
os → opérations système bas niveau
glob → recherche par pattern
zipfile → compression/décompression
watchdog → surveiller les changements en temps réel

— OPÉRATIONS DE BASE —

```
from pathlib import Path
import shutil

# Créer un dossier (et ses parents si besoin)
Path("data/output/rapports").mkdir(parents=True, exist_ok=True)

# Lister tous les CSV d'un dossier
csvs = list(Path("data/").glob("*.csv"))

# Lister récursivement tous les Excel
excels = list(Path("data/").rglob("*.xlsx"))

# Copier un fichier
shutil.copy("source.csv", "backup/source_backup.csv")

# Déplacer
shutil.move("temp/rapport.pdf", "output/rapport.pdf")

# Supprimer un dossier entier
shutil.rmtree("temp/", ignore_errors=True)

# Renommer avec horodatage
from datetime import datetime
ts = datetime.now().strftime("%Y%m%d_%H%M%S")
Path("rapport.pdf").rename(f"rapport_{ts}.pdf")
```

— LIRE ET ÉCRIRE DES FICHIERS TEXTE —

```

# Lire (toujours spécifier l'encodage)
with open("fichier.txt", "r", encoding="utf-8") as f:
    contenu = f.read()

# Écrire (écrase si existe)
with open("output.txt", "w", encoding="utf-8") as f:
    f.write(contenu)

# Ajouter sans écraser
with open("log.txt", "a", encoding="utf-8") as f:
    f.write(f"{datetime.now()} - nouvelle ligne\n")

# Lire ligne par ligne (économise la mémoire)
with open("gros_fichier.txt", "r", encoding="utf-8") as f:
    for line in f:
        process(line.strip())

— TRAVAILLER AVEC DES CSV —————

import csv

# Lire un CSV natif (sans pandas)
with open("data.csv", "r", encoding="utf-8") as f:
    reader = csv.DictReader(f)
    for row in reader:
        print(row["nom"], row["montant"])

# Écrire un CSV natif
rows = [{"nom": "Alice", "score": 95}, {"nom": "Bob", "score": 87}]
with open("output.csv", "w", newline="", encoding="utf-8") as f:
    writer = csv.DictWriter(f, fieldnames=["nom", "score"])
    writer.writeheader()
    writer.writerows(rows)

— TRAVAILLER AVEC DES FICHIERS EXCEL —————

import openpyxl

# Lire un Excel
wb = openpyxl.load_workbook("rapport.xlsx")
ws = wb.active
for row in ws.iter_rows(min_row=2, values_only=True):
    print(row)

# Écrire un Excel avec mise en forme
wb = openpyxl.Workbook()
ws = wb.active
ws.title = "Résultats"
ws.append(["Nom", "CA", "Marge"])
ws.append(["Client A", 10000, 0.25])
wb.save("output.xlsx")

— TRAVAILLER AVEC DES PDF —————

import pdfplumber

# Extraire le texte
with pdfplumber.open("document.pdf") as pdf:
    for page in pdf.pages:
        print(page.extract_text())

# Extraire les tableaux
with pdfplumber.open("rapport.pdf") as pdf:
    table = pdf.pages[0].extract_table()
    # table = liste de listes

# Créer un PDF simple
from fpdf import FPDF

pdf = FPDF()
pdf.add_page()
pdf.set_font("Arial", size=12)
pdf.cell(200, 10, txt="Rapport automatique", ln=True)
pdf.output("rapport_auto.pdf")

```

— SURVEILLER UN DOSSIER EN TEMPS RÉEL —

```
from watchdog.observers import Observer
from watchdog.events import FileSystemEventHandler

class MonHandler(FileSystemEventHandler):
    def on_created(self, event):
        if event.src_path.endswith(".csv"):
            print(f"Nouveau fichier détecté : {event.src_path}")
            process_file(event.src_path)

observer = Observer()
observer.schedule(MonHandler(), path="data/incoming/",
                 recursive=False)
observer.start()
# Le script tourne et réagit aux nouveaux fichiers
```

— ARCHIVAGE ET COMPRESSION —

```
import zipfile

# Compresser un dossier
with zipfile.ZipFile("archive.zip", "w",
                    zipfile.ZIP_DEFLATED) as zf:
    for fichier in Path("output/").rglob("*"):
        zf.write(fichier)

# Décompresser
with zipfile.ZipFile("archive.zip", "r") as zf:
    zf.extractall("destination/")
```

PIÈGES FICHIERS

- Toujours spécifier encoding="utf-8" à l'ouverture
- Utiliser pathlib plutôt que os.path (plus lisible)
- exist_ok=True dans mkdir pour éviter les erreurs
- Ne jamais modifier un fichier source directement, travailler sur une copie

A.2 – WEB ET APIS

LIBRAIRIES ESSENTIELLES

requests → appels HTTP (standard)
httpx → requests avec support async
beautifulsoup4 → parsing HTML
playwright → automatisation navigateur moderne

— APPELS API REST —

```
import requests
import os

# GET simple
response = requests.get(
    "https://api.example.com/data",
    headers={"Authorization": f"Bearer {os.getenv('API_KEY')}"},
    timeout=30
)
response.raise_for_status() # lève une erreur si 4xx/5xx
data = response.json()

# POST avec payload JSON
payload = {"nom": "Alice", "montant": 500}
response = requests.post(
    "https://api.example.com/orders",
    json=payload,
    headers={"Authorization": f"Bearer {os.getenv('API_KEY')}"})

# Télécharger un fichier
```

```

response = requests.get(url, stream=True)
with open("fichier.pdf", "wb") as f:
    for chunk in response.iter_content(chunk_size=8192):
        f.write(chunk)

```

— PAGINATION – NE JAMAIS L'IGNORER —

```

def get_all_pages(base_url, headers):
    """Récupère toutes les pages d'une API paginée."""
    results = []
    page = 1
    while True:
        response = requests.get(
            f"{base_url}?page={page}&per_page=100",
            headers=headers,
            timeout=30
        )
        response.raise_for_status()
        data = response.json()

        if not data or not data.get("results"):
            break

        results.extend(data["results"])
        page += 1

        # Respecter le rate limit
        time.sleep(0.5)

    return results

```

— RETRY AVEC BACKOFF —

```

import time
from requests.exceptions import RequestException

def api_call_with_retry(url, headers, max_retries=3):
    for attempt in range(max_retries):
        try:
            r = requests.get(url, headers=headers, timeout=30)
            r.raise_for_status()
            return r.json()
        except RequestException as e:
            if attempt == max_retries - 1:
                raise
            wait = 2 ** attempt
            print(f"Tentative {attempt+1} échouée. "
                  f"Retry dans {wait}s...")
            time.sleep(wait)

```

— SCRAPING HTML —

```

from bs4 import BeautifulSoup
import requests

response = requests.get("https://example.com", timeout=30)
soup = BeautifulSoup(response.text, "html.parser")

# Extraire par balise
titres = [h2.text.strip() for h2 in soup.find_all("h2")]

# Extraire par classe CSS
prix = soup.find_all("span", class_="price")

# Extraire un tableau HTML → DataFrame
import pandas as pd
tables = pd.read_html(response.text)
df = tables[0] # premier tableau trouvé

```

— SCRAPING NAVIGATEUR AVEC PLAYWRIGHT —

```

# Pour les pages qui nécessitent JavaScript
from playwright.sync_api import sync_playwright

with sync_playwright() as p:

```

```

browser = p.chromium.launch(headless=True)
page = browser.new_page()
page.goto("https://example.com")
page.wait_for_selector(".data-table")
contenu = page.inner_text(".data-table")
browser.close()

```

RÈGLES D'OR DU SCRAPING

- Vérifier robots.txt avant tout : /robots.txt
- Ajouter un délai entre les requêtes (time.sleep)
- Identifier ton scraper dans le User-Agent
- Vérifier si une API officielle existe (souvent oui)
- Gérer les timeouts et les pages vides
- Stocker la réponse brute avant de parser

A.3 – EMAILS ET NOTIFICATIONS

— ENVOYER UN EMAIL AVEC PIÈCE JOINTE —

```

import smtplib
from email.mime.multipart import MIMEMultipart
from email.mime.text import MIMEText
from email.mime.base import MIMEBase
from email import encoders
import os

def send_email(to, subject, body, attachment=None):
    msg = MIMEMultipart()
    msg["From"] = os.getenv("SMTP_FROM")
    msg["To"] = to
    msg["Subject"] = subject
    msg.attach(MIMEText(body, "html"))

    if attachment:
        with open(attachment, "rb") as f:
            part = MIMEBase("application", "octet-stream")
            part.set_payload(f.read())
            encoders.encode_base64(part)
            part.add_header("Content-Disposition",
                           f"attachment; filename={attachment}")
            msg.attach(part)

    with smtplib.SMTP(os.getenv("SMTP_HOST"), 587) as server:
        server.starttls()
        server.login(os.getenv("SMTP_USER"),
                    os.getenv("SMTP_PASS"))
        server.send_message(msg)

# Utilisation
send_email(
    to="client@exemple.com",
    subject="Rapport journalier – Ventes du jour",
    body="<h2>Bonjour,</h2><p>Voici le rapport ...</p>",
    attachment="rapport_2024.xlsx"
)

```

— LIRE SA BOÎTE EMAIL ET RÉAGIR —

```

import imaplib
import email

def fetch_unread_emails():
    with imaplib.IMAP4_SSL("imap.gmail.com") as mail:
        mail.login(os.getenv("EMAIL_USER"),
                  os.getenv("EMAIL_PASS"))
        mail.select("inbox")
        _, ids = mail.search(None, "UNSEEN")

        emails = []
        for uid in ids[0].split():
            _, msg_data = mail.fetch(uid, "(RFC822)")

```

```

        msg = email.message_from_bytes(msg_data[0][1])
        emails.append({
            "from": msg["From"],
            "subject": msg["Subject"],
            "date": msg["Date"]
        })
    return emails

```

— NOTIFICATIONS SLACK

```

import requests

def notify_slack(message, webhook_url=None):
    url = webhook_url or os.getenv("SLACK_WEBHOOK")
    requests.post(url, json={"text": message}, timeout=10)

# Utilisation
notify_slack("✅ Pipeline terminé – 1 247 lignes traitées")
notify_slack("❌ ERREUR pipeline ventes – voir logs")

```

— NOTIFICATIONS TELEGRAM

```

def notify_telegram(message):
    token = os.getenv("TELEGRAM_TOKEN")
    chat_id = os.getenv("TELEGRAM_CHAT_ID")
    url = f"https://api.telegram.org/bot{token}/sendMessage"
    requests.post(url, json={"chat_id": chat_id,
                             "text": message}, timeout=10)

```

A.4 – BASES DE DONNÉES

LIBRAIRIES

```

sqlite3    → SQLite intégré, parfait pour projets légers
sqlalchemy → ORM universel (PostgreSQL, MySQL, SQLite...)
psycopg2   → PostgreSQL natif
pandas     → lire/écrire SQL directement en DataFrame

```

— SQLITE – USAGE LOCAL

```

import sqlite3

# Connexion (crée le fichier si inexistant)
conn = sqlite3.connect("data.db")
cursor = conn.cursor()

# Créer une table
cursor.execute("""
    CREATE TABLE IF NOT EXISTS ventes (
        id        INTEGER PRIMARY KEY AUTOINCREMENT,
        date       TEXT,
        client     TEXT,
        montant    REAL
    )
""")

# Insérer des données (paramétré – jamais de f-string ici)
cursor.executemany(
    "INSERT INTO ventes (date, client, montant) VALUES (?, ?, ?)",
    [
        ("2024-01-15", "Alice", 1500.0),
        ("2024-01-15", "Bob", 800.0)
    ]
)
conn.commit()

# Lire
cursor.execute("SELECT * FROM ventes WHERE montant > ?", (1000,))
rows = cursor.fetchall()

conn.close()

```

— SQLALCHEMY – UNIVERSEL

```

from sqlalchemy import create_engine, text
import pandas as pd

# Connexion (modifier l'URL selon la BDD)
engine = create_engine("postgresql+psycopg2://"+
                        f"{os.getenv('DB_USER')}:"+
                        f"{os.getenv('DB_PASS')}@"+
                        f"{os.getenv('DB_HOST')}/"+
                        f"{os.getenv('DB_NAME')}")

# Lire directement en DataFrame
with engine.connect() as conn:
    df = pd.read_sql(
        text("SELECT * FROM ventes WHERE date >= :d"),
        conn,
        params={"d": "2024-01-01"}
    )

# Écrire un DataFrame en BDD
df.to_sql("ventes_clean", engine,
        if_exists="append", # ou "replace"
        index=False)

```

PATTERNS SQL UTILES

```

-- Upsert (INSERT si nouveau, UPDATE si existe)
INSERT INTO ventes (id, client, montant)
VALUES (?, ?, ?)
ON CONFLICT(id) DO UPDATE SET
    montant = excluded.montant;

```

```

-- Agrégats courants
SELECT client,
    COUNT(*)      as nb_commandes,
    SUM(montant)   as ca_total,
    AVG(montant)   as panier_moyen,
    MAX(date)      as derniere_commande
FROM ventes
GROUP BY client
ORDER BY ca_total DESC;

```

RÈGLES BDD

- Toujours utiliser des requêtes paramétrées (?, :param)
- Jamais de f-string dans une requête SQL
- Accès en lecture seule pendant le développement
- Tester les requêtes sur un échantillon avant le full run
- Logger le nombre de lignes insérées/mises à jour

A.5 – PLANIFICATION ET ORCHESTRATION

SCHEDULE – LÉGER, SANS DÉPENDANCES

```

import schedule
import time

def job_quotidien():
    print("Exécution du pipeline...")
    run_pipeline()

schedule.every().day.at("07:00").do(job_quotidien)
schedule.every().monday.at("09:00").do(rapport_hebdo)
schedule.every(30).minutes.do(check_api)

while True:
    schedule.run_pending()
    time.sleep(60)

```

CRON LINUX – POUR LA PRODUCTION

```

# Éditer avec : crontab -e
# Format : minute heure jour mois jour_semaine commande

```

```
# Tous les jours à 7h
0 7 * * * /projet/venv/bin/python /projet/main.py >> /projet/logs/cron.log 2>&1

# Tous les lundis à 9h
0 9 * * 1 /projet/venv/bin/python /projet/rapport_hebdo.py >> /projet/logs/cron.log 2>&1

# Toutes les 30 minutes
*/30 * * * * /projet/venv/bin/python /projet/check.py
```

— MESURER LA DURÉE D'EXÉCUTION —

```
import time

start = time.time()
run_pipeline()
elapsed = time.time() - start
logger.info(f"Durée : {elapsed:.1f}s ({elapsed/60:.1f} min)")
```

PARTIE B – ANALYSE DE DONNÉES

B.1 – CHARGEMENT ET PREMIÈRE EXPLORATION

```
import pandas as pd
import numpy as np
```

— CHARGER LES DONNÉES —

```
# CSV
df = pd.read_csv("data.csv", encoding="utf-8", sep=";",
                 parse_dates=["date"],
                 dtype={"id": str, "code": str})

# Excel
df = pd.read_excel("data.xlsx", sheet_name="Ventes",
                  skiprows=1) # si ligne parasite en tête

# JSON
df = pd.read_json("data.json")

# SQL
df = pd.read_sql("SELECT * FROM ventes", engine)

# Gros fichier en chunks
chunks = []
for chunk in pd.read_csv("big.csv", chunksize=50000):
    chunks.append(process(chunk))
df = pd.concat(chunks, ignore_index=True)
```

— EXPLORATION INITIALE – TOUJOURS DANS CET ORDRE —

```
# 1. Dimensions
print(df.shape) # (nb_lignes, nb_colonnes)

# 2. Aperçu
print(df.head(10))
print(df.tail(5))

# 3. Types et non-nuls
print(df.info())

# 4. Statistiques descriptives
print(df.describe(include="all"))

# 5. Valeurs manquantes
print(df.isnull().sum())
print(df.isnull().mean().round(3) * 100) # en %

# 6. Doublons
```



```
print(f"Doublons : {df.duplicated().sum()}")
print(f"Doublons sur id : {df.duplicated('id').sum()}")

# 7. Valeurs uniques des catégorielles
for col in df.select_dtypes(include="object").columns:
    print(f"{col} : {df[col].nunique()} valeurs uniques")
    if df[col].nunique() < 20:
        print(df[col].value_counts())

# 8. Distribution des numériques
print(df.describe(percentiles=[.05, .25, .5, .75, .95]))
```

— QUESTIONS À SE POSER À L'EXPLORATION —

- Quelle est la granularité ? (1 ligne = 1 quoi ?)
- Y a-t-il une clé primaire unique ?
- Quelles colonnes sont des identifiants, des dates, des mesures, des catégories ?
- Le volume est-il cohérent avec le contexte métier ?
- Quelles colonnes auront le plus de valeur analytique ?

B.2 – NETTOYAGE DES DONNÉES

— VALEURS MANQUANTES —

```
# Supprimer si trop de manquants (>50%)
threshold = len(df) * 0.5
df = df.dropna(thresh=threshold, axis=1)

# Supprimer les lignes sans la colonne clé
df = df.dropna(subset=["id", "date"])

# Remplacer – TOUJOURS valider le choix avec le métier
df["montant"] = df["montant"].fillna(0) # si NaN = 0
df["categorie"] = df["categorie"].fillna("INCONNU")
df["montant"] = df["montant"].fillna(
    df["montant"].median() # si NaN = médiane
```

— DOUBLONS —

```
# Voir les doublons
doublons = df[df.duplicated(keep=False)]

# Supprimer (garder le premier)
df = df.drop_duplicates(subset=["id"])

# Supprimer en gardant le plus récent
df = (df.sort_values("date", ascending=False)
      .drop_duplicates(subset=["id"])
      .sort_index())
```

— TYPES DE DONNÉES —

```
# Dates
df["date"] = pd.to_datetime(df["date"],
                           format="%Y-%m-%d",
                           errors="coerce") # NaT si invalide
dates_invalides = df["date"].isna().sum()

# Numériques avec caractères parasites
df["montant"] = (df["montant"]
                 .astype(str)
                 .str.replace("€", "", regex=False)
                 .str.replace(",", ".", regex=False)
                 .str.strip()
                 .pipe(pd.to_numeric, errors="coerce"))

# Catégorielles (économise la mémoire)
df["statut"] = df["statut"].astype("category")
```

— NETTOYAGE DE TEXTE —

```
# Pipeline de nettoyage texte
df["nom"] = (df["nom"]
             .astype(str)
             .str.strip()
             .str.upper()
             .str.replace(r"\s+", " ", regex=True))

# Normaliser les variantes
mapping = {
    "paris": "Paris", "PARIS": "Paris", "paris  ": "Paris",
    "lyon": "Lyon", "LYON": "Lyon"
}
df["ville"] = df["ville"].str.strip().map(
    lambda x: mapping.get(x, x) if pd.notna(x) else x
)
```

— OUTLIERS —

```
# Méthode IQR (robuste)
Q1 = df["montant"].quantile(0.25)
Q3 = df["montant"].quantile(0.75)
IQR = Q3 - Q1
borne_basse = Q1 - 1.5 * IQR
borne_haute = Q3 + 1.5 * IQR

outliers = df[(df["montant"] < borne_basse) |
               (df["montant"] > borne_haute)]
print(f"{len(outliers)} outliers détectés")

# NE PAS supprimer sans validation métier
# Créer un flag plutôt que de supprimer
df["is_outlier"] = (
    (df["montant"] < borne_basse) |
    (df["montant"] > borne_haute)
)
```

B.3 – TRANSFORMATION ET FEATURE ENGINEERING

— DÉRIVÉES DE DATES —

```
df["annee"]      = df["date"].dt.year
df["mois"]       = df["date"].dt.month
df["semaine"]    = df["date"].dt.isocalendar().week
df["jour_sem"]   = df["date"].dt.day_name()
df["trimestre"]  = df["date"].dt.quarter
df["is_weekend"] = df["date"].dt.dayofweek >= 5
df["heure"]      = df["date"].dt.hour

# Différence entre deux dates
df["jours_depuis_inscription"] = (
    pd.Timestamp.now() - df["date_inscription"]
).dt.days
```

— AGRÉGATIONS —

```
# Agrégation simple
resume = df.groupby("client_id").agg(
    nb_commandes      = ("id",      "count"),
    ca_total           = ("montant", "sum"),
    panier_moyen       = ("montant", "mean"),
    premiere_commande  = ("date",    "min"),
    derniere_commande  = ("date",    "max")
).reset_index()

# Agrégation multi-niveaux
resume_mensuel = df.groupby(
    ["client_id", df["date"].dt.to_period("M")]
)["montant"].sum().reset_index()

# Cumul
df = df.sort_values(["client_id", "date"])
df["ca_cumul"] = df.groupby("client_id")["montant"].cumsum()
```

— JOINTURES —

```
# Toujours vérifier avant de joindre
print(f"Clés uniques gauche : {df1['id'].nunique()}")
print(f"Clés uniques droite : {df2['id'].nunique()}")
print(f"Intersection : "
      f"{df1['id'].isin(df2['id']).sum()}")

# Jointure avec vérification du résultat
n_avant = len(df1)
df = df1.merge(df2, on="id", how="left")
n_apres = len(df)
if n_apres != n_avant:
    print(f"ATTENTION : {n_apres - n_avant} lignes "
          f"en plus après la jointure (doublons côté droite ?)")
```

— NOUVELLES VARIABLES UTILES —

```
# Segmentation RFM simplifiée
now = pd.Timestamp.now()
rfm = df.groupby("client_id").agg(
    recence      = ("date",      lambda x: (now - x.max()).days),
    frequence     = ("id",       "count"),
    valeur        = ("montant",  "sum")
)

# Score par percentile
for col in ["recence", "frequence", "valeur"]:
    rfm[f"{col}_score"] = pd.qcut(rfm[col], q=4,
                                  labels=[1, 2, 3, 4])
```

B.4 – ANALYSE STATISTIQUE

— STATISTIQUES DESCRIPTIVES AVANCÉES —

```
# Distribution complète
stats = {
    "moyenne" : df["montant"].mean(),
    "mediane" : df["montant"].median(),
    "ecart_type": df["montant"].std(),
    "skewness" : df["montant"].skew(),
    "kurtosis" : df["montant"].kurt(),
    "p5"       : df["montant"].quantile(0.05),
    "p95"      : df["montant"].quantile(0.95)
}

# Concentration (règle des 80/20)
df_sorted = df.sort_values("montant", ascending=False)
df_sorted["ca_cum_pct"] = (
    df_sorted["montant"].cumsum() / df_sorted["montant"].sum()
)
n_80_pct = (df_sorted["ca_cum_pct"] <= 0.8).sum()
total = len(df_sorted)
print(f"{n_80_pct} clients ({n_80_pct/total*100:.0f}%) "
      f"représentent 80% du CA")
```

— CORRÉLATIONS —

```
# Matrice de corrélation
corr = df.select_dtypes(include="number").corr()

# Corrélations fortes avec une variable cible
target_corr = corr["montant"].abs().sort_values(ascending=False)
print(target_corr)
```

— ÉVOLUTION TEMPORELLE —

```
# Croissance période sur période
mensuel = df.groupby(df["date"].dt.to_period("M"))["montant"].sum()
mensuel_df = mensuel.reset_index()
mensuel_df["croissance_pct"] = mensuel_df["montant"].pct_change() * 100
```

```
# Moyenne mobile (lissage)
mensuel_df["mm_3m"] = mensuel_df["montant"].rolling(3).mean()
```

— SEGMENTATION SIMPLE —

```
# Segmentation par quartile
df["segment_valeur"] = pd.qcut(
    df["ca_total"],
    q=4,
    labels=["Bronze", "Argent", "Or", "Platine"]
)

# Segmentation avec règles métier
conditions = [
    df["ca_total"] >= 10000,
    df["ca_total"] >= 5000,
    df["ca_total"] >= 1000,
]
labels = ["Grand compte", "Compte moyen", "Petit compte"]
df["segment"] = np.select(conditions, labels,
                          default="Prospect")
```

B.5 – VISUALISATION

```
import matplotlib.pyplot as plt
import matplotlib.ticker as mticker

# Style propre pour les livrables client
plt.rcParams.update({
    "figure.figsize" : (12, 6),
    "axes.spines.top" : False,
    "axes.spines.right" : False,
    "font.family" : "sans-serif",
    "axes.titlesize" : 14,
    "axes.titleweight": "bold"
})
```

— GRAPHIQUES PAR TYPE DE QUESTION —

ÉVOLUTION DANS LE TEMPS → Courbe

```
fig, ax = plt.subplots()
ax.plot(mensuel_df["date"].astype(str),
        mensuel_df["montant"] / 1000,
        marker="o", linewidth=2, color="#2563EB")
ax.fill_between(range(len(mensuel_df)),
                mensuel_df["montant"] / 1000, alpha=0.1,
                color="#2563EB")
ax.set_title("CA mensuel – Évolution sur 12 mois")
ax.set_ylabel("CA (k€)")
ax.yaxis.set_major_formatter(mticker.FormatStrFormatter("%.0fk€"))
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("output/ca_mensuel.png", dpi=150)
plt.close()
```

COMPARAISON DE CATÉGORIES → Barres horizontales

```
top10 = df.nlargest(10, "ca_total")
fig, ax = plt.subplots()
bars = ax.barh(top10["client"], top10["ca_total"] / 1000,
               color="#2563EB")
ax.bar_label(bars, fmt="%.0fk€", padding=4)
ax.set_title("Top 10 clients par CA")
ax.invert_yaxis()
plt.tight_layout()
plt.savefig("output/top10_clients.png", dpi=150)
plt.close()
```

DISTRIBUTION → Histogramme

```
fig, ax = plt.subplots()
ax.hist(df["montant"], bins=30, color="#2563EB",
        edgecolor="white")
ax.axvline(df["montant"].median(), color="#DC2626",
           linestyle="--", label=f"Médiane : {df['montant'].median():.0f}€")
ax.set_title("Distribution des montants de commande")
ax.legend()
plt.tight_layout()
plt.savefig("output/distribution_montants.png", dpi=150)
plt.close()
```

PARTS → Barres empilées (préférer aux camemberts)

```
pivot = df.groupby(["mois", "categorie"])["montant"].sum().unstack()
pivot.plot(kind="bar", stacked=True, figsize=(12, 6))
plt.title("Répartition du CA par catégorie et par mois")
plt.tight_layout()
plt.savefig("output/repartition_ca.png", dpi=150)
plt.close()
```

RÈGLES DE VISUALISATION POUR LIVRABLES CLIENT

- Titre = conclusion, pas description
Bon : "Les ventes ont baissé de 23% en Q3"
Mauvais : "Ventes Q3 par catégorie"
- Une seule couleur principale + une couleur d'accent
- Supprimer les grilles trop chargées
- Annoter les valeurs clés directement sur le graphique
- Toujours indiquer l'unité sur les axes
- Sauvegarder en PNG 150dpi minimum pour les rapports

B.6 – MODÉLISATION

```
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import (
    mean_absolute_error, r2_score,
    classification_report, confusion_matrix
)
```

— PIPELINE DE MODÉLISATION STANDARD —

```
# 1. Préparer X et y
features = ["recence", "frequence", "valeur", "nb_produits"]
target = "a_churne"

X = df[features]
y = df[target]

# 2. Séparer train / test (TOUJOURS avant tout preprocessing)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42,
    stratify=y # pour la classification
)

# 3. Preprocessing
scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test) # PAS fit ici

# 4. Modèle – commencer simple
from sklearn.ensemble import RandomForestClassifier
model = RandomForestClassifier(n_estimators=100,
                              random_state=42,
                              n_jobs=-1)

model.fit(X_train_s, y_train)

# 5. Évaluation
y_pred = model.predict(X_test_s)
print(classification_report(y_test, y_pred))

# 6. Importance des variables
```

```
importances = pd.Series(model.feature_importances_,
                        index=features).sort_values(
                        ascending=False)

print(importances)
```

— MODÈLES PAR TYPE DE PROBLÈME —

RÉGRESSION (prédire un nombre)

- LinearRegression : baseline, interprétable
- Ridge / Lasso : si multicollinéarité
- RandomForestRegressor : si non-linéaire
- XGBRegressor : si haute performance requise

Métriques : MAE, RMSE, R²

CLASSIFICATION (prédire une catégorie)

- LogisticRegression : baseline, interprétable
- RandomForestClassifier: robuste, peu de réglage
- XGBClassifier : si haute performance requise

Métriques : Accuracy, Précision, Rappel, F1, AUC-ROC
 Attention : classes déséquilibrées → class_weight="balanced"

CLUSTERING (segmenter sans label)

- KMeans : si on connaît le nombre de groupes
- DBSCAN : si on ne le connaît pas
- Agglomerative : si on veut une hiérarchie

Évaluation : silhouette score, cohérence métier

SÉRIE TEMPORELLE (prédire le futur)

- Prophet (Meta) : simple, interprétable, saisonnalité
- statsmodels SARIMA : si modèle statistique classique
- XGBoost + features temporelles : si haute performance

Métriques : MAPE, MAE, RMSE

— ERREURS CLASSIQUES DE MODÉLISATION —

Data Leakage

- Utiliser des infos du futur dans les features d'entraînement
- Scaler AVANT le split → contamination
- Feature calculée sur l'ensemble entier

Overfitting non détecté

- Score train >> score test → modèle qui mémorise
- Solution : cross-validation, régularisation, plus de données

Mauvaise métrique

- Optimiser l'accuracy sur données déséquilibrées
- Solution : F1, AUC, précision/rappel selon le cas métier

Interprétation causale

- Corrélation ≠ causalité
- Un modèle prédit, il n'explique pas sans analyse approfondie

B.7 – RAPPORT AUTOMATIQUE

"La dernière étape : transformer les résultats en décisions."

— GÉNÉRER UN RAPPORT EXCEL AUTOMATIQUE —

```
with pd.ExcelWriter("rapport_auto.xlsx",
                    engine="openpyxl") as writer:
    df_resume.to_excel(writer, sheet_name="Résumé", index=False)
    df_detail.to_excel(writer, sheet_name="Détail", index=False)
    df_top10.to_excel(writer, sheet_name="Top clients", index=False)
```

— RAPPORT HTML SIMPLE —

```
html = f"""
<html><body>
<h1>Rapport automatique – {datetime.now().strftime('%d/%m/%Y')}</h1>
<h2>Résumé</h2>
<p>CA total : <strong>{ca_total:,.0f} €</strong></p>
<p>Nb commandes : <strong>{nb_commandes}</strong></p>
<h2>Top 10 clients</h2>
```

```
{df_top10.to_html(index=False, float_format="{:.0f}".format)}
</body></html>
"""
with open("rapport.html", "w", encoding="utf-8") as f:
    f.write(html)
```

— DASHBOARD STREAMLIT EN 20 LIGNES —

```
import streamlit as st

st.set_page_config(page_title="Dashboard Ventes", layout="wide")
st.title("Dashboard Ventes")

col1, col2, col3 = st.columns(3)
col1.metric("CA Total", f"{ca_total:,.0f} €")
col2.metric("Nb Commandes", nb_commandes)
col3.metric("Panier Moyen", f"{panier_moyen:,.0f} €")

st.subheader("Évolution mensuelle")
st.line_chart(mensuel_df.set_index("date")["montant"])

st.subheader("Top 10 clients")
st.dataframe(df_top10, use_container_width=True)
```

RÉFÉRENCE RAPIDE – QUELLE LIBRAIRIE POUR QUEL BESOIN

BESOIN	LIBRAIRIE(S)
Fichiers CSV / Excel	pandas, openpyxl, csv
Fichiers PDF	pdfplumber, fpdf2
Base de données	sqlalchemy, sqlite3, psycopg2
Appels API REST	requests, httpx
Scraping HTML	beautifulsoup4, playwright
Emails (envoyer)	smtplib, yagmail
Emails (lire)	imaplib
Notifications Slack	requests (webhook)
Notifications Telegram	python-telegram-bot
Planification	schedule, cron
Surveiller un dossier	watchdog
Nettoyage et manipulation	pandas, numpy
Visualisation statique	matplotlib, seaborn
Visualisation interactive	plotly
Machine Learning	scikit-learn, xgboost
Séries temporelles	prophet, statsmodels
Dashboard	streamlit, metabase
Logging	loguru
Configuration .env	python-dotenv
Interface CLI	click, argparse
Tests	pytest
Compression	zipfile, shutil

CHECKLIST PROJET COMPLET – À COCHER AVANT LIVRAISON

DONNÉES

- ☐ Source brute conservée et non modifiée
- ☐ Chaque décision de nettoyage documentée et validée
- ☐ Valeurs manquantes traitées avec justification métier
- ☐ Doublons vérifiés et gérés
- ☐ Types de données corrects dans toutes les colonnes

AUTOMATISATION

- ☐ Credentials dans .env, jamais dans le code
- ☐ Gestion d'erreurs sur toutes les étapes critiques
- ☐ Logging activé avec résumé d'exécution
- ☐ Script idempotent (peut tourner 2x sans corruption)
- ☐ Planification configurée et testée

ANALYSE

- ☐ Les résultats ont été vérifiés contre une valeur connue

- [] Les graphiques ont des titres = conclusions
- [] L'unité est indiquée sur tous les axes
- [] Les outliers sont signalés, pas ignorés
- [] Le modèle a été évalué sur des données non vues

LIVRAISON

- [] README complet et testé
- [] requirements.txt avec versions figées
- [] Le client peut utiliser la solution sans aide
- [] Recette signée
- [] Facture envoyée

« Les données te diront la vérité si tu sais
poser les bonnes questions.
Et les bonnes questions viennent toujours
du terrain, jamais du terminal. »
